

INFORME DE ESPECIFICACIÓN DE DATOS Y BASE DE DATOS

Plataforma Web de Intermediación de Servicios u Oficios Informales

Elemento	Especificación
Backend	NestJS — API RESTful
ORM	Prisma
Base de datos	PostgreSQL
Frontend	Angular 17+
Autenticación	JWT + Bcrypt
Validación	class-validator + ValidationPipe
Documentación	Swagger / OpenAPI
Versión	1.0 — Agosto 2026

Objetivo: definir entidades, campos, relaciones, reglas de negocio, seguridad, índices, consultas y arquitectura de datos para el Backend NestJS.

Índice

1. Resumen ejecutivo
2. Alcance de la información
3. Arquitectura general de datos
4. Entidades y modelo conceptual
5. Diccionario de datos
6. Relaciones y cardinalidades
7. Integridad y reglas de negocio
8. Seguridad y autenticación
9. Consultas y filtros
10. Índices y rendimiento
11. API REST propuesta
12. Estructura NestJS
13. Distribución Backend 1 / Backend 2
14. Modelo Prisma base
15. Migraciones y seed
16. Datos fuera de alcance
17. Extensiones futuras
18. Casos de uso
19. Checklist
20. Conclusiones y recomendaciones

1. Resumen ejecutivo

La plataforma conecta clientes que necesitan servicios con trabajadores independientes que ofrecen oficios. Facilita descubrimiento, publicación, contacto y reputación, pero no procesa pagos ni incorpora chat interno.

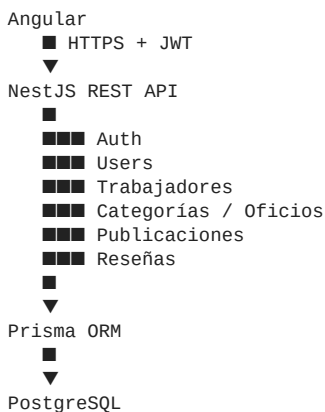
El núcleo de datos está compuesto por usuarios, perfiles de trabajadores, categorías, oficios, relación trabajador-oficio, publicaciones y reseñas. Se recomienda PostgreSQL + Prisma por la cantidad de relaciones y por su integración con TypeScript/NestJS.

2. Alcance de la información

- Usuarios y autenticación.
- Roles CLIENTE y TRABAJADOR.
- Perfil profesional, experiencia, disponibilidad y zona.
- Hasta tres oficios principales por trabajador.
- Categorías y catálogo de oficios.
- Publicaciones y estados ABIERTA/EN_PROGRESO/COMPLETADA.
- Presupuesto opcional.
- Reseñas de 1 a 5 estrellas.
- Datos para WhatsApp y llamadas.
- Refresh tokens, si se implementan.

Fuera del alcance: pagos, tarjetas, chat, mensajes internos, comisiones, facturación, PWA/offline y sincronización local.

3. Arquitectura general de datos



Regla arquitectónica: Angular no accede directamente a PostgreSQL. Los controllers reciben peticiones, los services contienen reglas de negocio y Prisma gestiona persistencia.

4. Entidades y modelo conceptual



■■■ N:1 ■■ OFICIO
■■■ 1:0..1 ■■ RESEÑA

La relación N:M se resuelve con trabajador_oficios, que además guarda si un oficio es principal. Esto permite implementar el requisito de máximo tres oficios principales.

5. Diccionario de datos

5.1 usuarios

Campo	Tipo	Nulo	Regla	Descripción
id	UUID	No	PK	Identificador.
nombre	VARCHAR(100)	No	—	Nombre.
apellido	VARCHAR(100)	No	—	Apellido.
email	VARCHAR(150)	No	UNIQUE	Correo.
telefono	VARCHAR(20)	No	UNIQUE	Número.
password_hash	VARCHAR(255)	No	—	Hash Bcrypt.
rol	ENUM	No	CLIENTE/TRABAJADOR	Rol.
foto_url	TEXT	Sí	—	Foto.
activo	BOOLEAN	No	DEFAULT true	Cuenta.
created_at	TIMESTAMP	No	DEFAULT now()	Creación.
updated_at	TIMESTAMP	No	DEFAULT now()	Actualización.

Email y teléfono deben ser únicos. Nunca se almacena ni se devuelve la contraseña original.

5.2 perfiles_trabajador

Campo	Tipo	Nulo	Regla	Descripción
id	UUID	No	PK	Identificador.
usuario_id	UUID	No	UNIQUE/FK	Usuario.
descripcion	TEXT	Sí	—	Presentación.
experiencia	TEXT	Sí	—	Experiencia.
zona_cobertura	VARCHAR(150)	No	—	Zona.
disponible	BOOLEAN	No	DEFAULT true	Disponibilidad.
created_at	TIMESTAMP	No	DEFAULT now()	Creación.
updated_at	TIMESTAMP	No	DEFAULT now()	Actualización.

Solo usuarios con rol TRABAJADOR deben tener este perfil.

5.3 categorias

Campo	Tipo	Nulo	Regla	Descripción
id	UUID	No	PK	Identificador.
nombre	VARCHAR(100)	No	UNIQUE	Nombre.
descripcion	TEXT	Sí	—	Descripción.
activo	BOOLEAN	No	DEFAULT true	Estado.
created_at	TIMESTAMP	No	DEFAULT now()	Creación.

5.4 oficios

Campo	Tipo	Nulo	Regla	Descripción
id	UUID	No	PK	Identificador.
categoria_id	UUID	No	FK	Categoría.
nombre	VARCHAR(100)	No	UNIQUE	Nombre.

Campo	Tipo	Nulo	Regla	Descripción
descripcion	TEXT	Sí	—	Descripción.
activo	BOOLEAN	No	DEFAULT true	Estado.
created_at	TIMESTAMP	No	DEFAULT now()	Creación.

5.5 trabajador_oficios

Campo	Tipo	Nulo	Regla	Descripción
trabajador_id	UUID	No	PK/FK	Perfil.
oficio_id	UUID	No	PK/FK	Oficio.
principal	BOOLEAN	No	DEFAULT false	Principal.
created_at	TIMESTAMP	No	DEFAULT now()	Asociación.

PK compuesta: (trabajador_id, oficio_id). Máximo tres principales mediante validación del service.

5.6 publicaciones

Campo	Tipo	Nulo	Regla	Descripción
id	UUID	No	PK	Identificador.
cliente_id	UUID	No	FK	Cliente.
oficio_id	UUID	No	FK	Oficio.
titulo	VARCHAR(150)	No	—	Título.
descripcion	TEXT	No	—	Detalle.
ubicacion	VARCHAR(200)	No	—	Lugar.
presupuesto	DECIMAL(10,2)	Sí	>=0	Opcional.
estado	ENUM	No	3 estados	Ciclo.
created_at	TIMESTAMP	No	DEFAULT now()	Creación.
updated_at	TIMESTAMP	No	DEFAULT now()	Actualización.

cliente_id se obtiene del JWT. Una publicación nueva inicia como ABIERTA.

5.7 reseñas

Campo	Tipo	Nulo	Regla	Descripción
id	UUID	No	PK	Identificador.
cliente_id	UUID	No	FK	Autor.
trabajador_id	UUID	No	FK	Evaluated.
publicacion_id	UUID	No	FK/UNIQUE*	Trabajo.
calificacion	SMALLINT	No	1..5	Puntuación.
comentario	TEXT	Sí	—	Comentario.
created_at	TIMESTAMP	No	DEFAULT now()	Fecha.

* Se recomienda una reseña por publicación.

5.8 refresh_tokens

Campo	Tipo	Nulo	Descripción
id	UUID	No	Identificador.
usuario_id	UUID	No	Usuario.

Campo	Tipo	Nulo	Descripción
token_hash	VARCHAR(255)	No	Hash.
expires_at	TIMESTAMP	No	Expiración.
revoked_at	TIMESTAMP	Sí	Revocación.
created_at	TIMESTAMP	No	Creación.

Recomendado para sesiones revocables; puede omitirse si el diseño final usa JWT de corta duración.

6. Relaciones y cardinalidades

Relación	Cardinalidad	Explicación
Usuario → Perfil	1:1	Un trabajador posee un perfil.
Categoría → Oficios	1:N	Agrupar oficios.
Trabajador → Oficios	N:M	Mediante trabajador_oficios.
Cliente → Publicaciones	1:N	Varias solicitudes.
Oficio → Publicaciones	1:N	Muchas solicitudes.
Cliente → Reseñas	1:N	Varias evaluaciones.
Trabajador → Reseñas	1:N	Varias evaluaciones recibidas.
Publicación → Reseña	1:0..1	Recomendado.
Usuario → RefreshTokens	1:N	Varias sesiones.

Decisión sobre contratación

Los requisitos no definen una asignación formal de un trabajador a una publicación. No se recomienda inventar una tabla de contratación en la primera versión. Si posteriormente se necesita registrar quién tomó un trabajo, crear asignaciones o servicios_contratados.

7. Integridad y reglas de negocio

ID	Regla
RN-01	Un usuario tiene un único rol.
RN-02	Solo trabajadores tienen perfil profesional.
RN-03	Máximo 3 oficios principales.
RN-04	Solo clientes crean publicaciones.
RN-05	Toda publicación pertenece a un oficio.
RN-06	Nueva publicación = ABIERTA.
RN-07	Solo el propietario modifica su publicación.
RN-08	COMPLETADA no se edita libremente.
RN-09	Calificación 1..5.
RN-10	Un cliente no puede calificarse como trabajador.
RN-11	Identidad obtenida del JWT.
RN-12	Password siempre Bcrypt.
RN-13	Presupuesto NULL = no especificado.
RN-14	Promedio calculable con AVG().
RN-15	Preferir baja lógica de catálogos.

8. Seguridad y autenticación

Registro:
POST /auth/register → DTO → unicidad → Bcrypt → Usuario
→ PerfilTrabajador si corresponde → tokens.

Login:
POST /auth/login → buscar → comparar Bcrypt → JWT.

Petición protegida:
JWT → JwtAuthGuard → RolesGuard → Controller → Service → Prisma → PostgreSQL.

Aplicar ValidationPipe global y class-validator. Los endpoints sensibles deben validar autenticación, rol y propiedad del recurso. Nunca devolver password_hash.

9. Consultas y filtros

Consulta	Filtros
Trabajadores	oficioid, categorioid, zona, disponible
Publicaciones	oficioid, categorioid, ubicación, estado
Feed	estado, oficio, ubicación, paginación
Perfil	trabajadorId
Reputación	trabajadorId
Mis publicaciones	clienteld desde JWT

Las listas deben paginarse. page/limit es suficiente para la primera versión.

10. Índices y rendimiento

Tabla	Índices
usuarios	UNIQUE(email), UNIQUE(telefono), INDEX(rol)
perfiles_trabajador	INDEX(zona_cobertura), INDEX(disponible)
categorias	UNIQUE(nombre), INDEX(activo)
oficios	UNIQUE(nombre), INDEX(categoria_id), INDEX(activo)
trabajador_oficios	INDEX(oficio_id), INDEX(trabajador_id)
publicaciones	INDEX(cliente_id), INDEX(oficio_id), INDEX(estado), INDEX(ubicacion), INDEX(created_at)
resenas	INDEX(trabajador_id), INDEX(cliente_id), INDEX(publicacion_id)
refresh_tokens	INDEX(usuario_id), INDEX(expires_at)

El objetivo de menos de 300 ms depende también del hosting, volumen, red, paginación y consultas. Estos índices son la base para evitar full scans en las operaciones frecuentes.

11. API REST propuesta

11.1 Auth

Método	Endpoint	Descripción	Acceso
POST	/auth/register	Crear cuenta	Público
POST	/auth/login	Iniciar sesión	Público
POST	/auth/refresh	Renovar	Token
POST	/auth/logout	Revocar	Autenticado

11.2 Usuarios y trabajadores

Método	Endpoint	Descripción
GET	/users/me	Perfil autenticado
PATCH	/users/me	Editar datos
GET	/trabajadores	Buscar con filtros
GET	/trabajadores/:id	Perfil público
PATCH	/trabajadores/me	Editar profesional
PUT	/trabajadores/me/oficios	Gestionar oficios

11.3 Catálogo, publicaciones y reseñas

Método	Endpoint	Descripción
GET	/categorias	Categorías activas
GET	/oficios	Oficios y filtros
POST	/publicaciones	Crear solicitud
GET	/publicaciones	Feed
GET	/publicaciones/:id	Detalle
PATCH	/publicaciones/:id	Editar/estado
DELETE	/publicaciones/:id	Eliminar propia
POST	/resenas	Crear reseña

Método	Endpoint	Descripción
GET	/trabajadores/:id/resenas	Reseñas
GET	/trabajadores/:id/resumen	Resumen

12. Estructura NestJS

```
src/
├── auth/
├── users/
├── trabajadores/
├── categorias/
├── oficios/
├── publicaciones/
├── resenas/
├── prisma/
│   ├── prisma.service.ts
│   ├── prisma.module.ts
│   └── common/
│       ├── decorators/
│       ├── guards/
│       ├── filters/
│       └── pipes/
├── app.module.ts
└── main.ts
```

Cada dominio debe tener controller, service, module y DTOs. Los controllers no deben contener consultas Prisma complejas.

13. Distribución Backend 1 / Backend 2

Equipo	Responsabilidades	Tablas
Backend 1 — Core & Auth	NestJS, Prisma, migraciones, JWT, Bcrypt, usuarios, perfiles, roles, Swagger	usuarios, perfiles_trabajador, refresh_tokens
Backend 2 — Negocio	categorías, oficios, publicaciones, filtros, reseñas	categorias, oficios, trabajador_oficios, publicaciones, resenas

schema.prisma, enums, DTOs, JWT, errores y paginación deben acordarse como contrato común antes de integrar.

14. Modelo Prisma base

```
enum Role {
  CLIENTE
  TRABAJADOR
}

enum PublicacionEstado {
  ABIERTA
  EN_PROGRESO
  COMPLETADA
}

model Usuario {
  id String @id @default(uuid())
  nombre String
  apellido String
  email String @unique
  telefono String @unique
  passwordHash String
  rol Role
  fotoUrl String?
  activo Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  perfilTrabajador PerfilTrabajador?
  publicaciones Publicacion[]
  resenasComoCliente Resena[] @relation("ResenasCliente")
  resenasComoTrabajador Resena[] @relation("ResenasTrabajador")
  refreshToken RefreshToken[]
}

model PerfilTrabajador {
  id String @id @default(uuid())
  usuarioId String @unique
  descripcion String?
  experiencia String?
  zonaCobertura String
  disponible Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  usuario Usuario @relation(fields: [usuarioId], references: [id])
  oficios TrabajadorOficio[]
}

model Categoria {
  id String @id @default(uuid())
  nombre String @unique
  descripcion String?
  activo Boolean @default(true)
  createdAt DateTime @default(now())
  oficios Oficio[]
}

model Oficio {
  id String @id @default(uuid())
  categoriaId String
  nombre String @unique
  descripcion String?
  activo Boolean @default(true)
  createdAt DateTime @default(now())
  categoria Categoria @relation(fields: [categoriaId], references: [id])
  trabajadores TrabajadorOficio[]
  publicaciones Publicacion[]
}

model TrabajadorOficio {
  trabajadorId String
  oficioId String
  principal Boolean @default(false)
  createdAt DateTime @default(now())
  trabajador PerfilTrabajador @relation(fields: [trabajadorId], references: [id])
  oficio Oficio @relation(fields: [oficioId], references: [id])
  @@id([trabajadorId, oficioId])
  @@index([oficioId])
}

model Publicacion {
  id String @id @default(uuid())
  clienteId String
```

```

oficioId String
titulo String
descripcion String
ubicacion String
presupuesto Decimal?
estado PublicacionEstado @default(ABIERTA)
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
cliente Usuario @relation(fields: [clienteId], references: [id])
oficio Oficio @relation(fields: [oficioId], references: [id])
resena Resena?
@@index([clienteId])
@@index([oficioId])
@@index([estado])
@@index([ubicacion])
@@index([createdAt])
}

model Resena {
  id String @id @default(uuid())
  clienteId String
  trabajadorId String
  publicacionId String @unique
  calificacion Int
  comentario String?
  createdAt DateTime @default(now())
  cliente Usuario @relation("ResenasCliente", fields: [clienteId], references: [id])
  trabajador Usuario @relation("ResenasTrabajador", fields: [trabajadorId], references: [id])
  publicacion Publicacion @relation(fields: [publicacionId], references: [id])
  @@index([trabajadorId])
  @@index([clienteId])
}

model RefreshToken {
  id String @id @default(uuid())
  usuarioId String
  tokenHash String
  expiresAt DateTime
  revokedAt DateTime?
  createdAt DateTime @default(now())
  usuario Usuario @relation(fields: [usuarioId], references: [id])
  @@index([usuarioId])
  @@index([expiresAt])
}

```

La regla de máximo tres oficios principales debe validarse en el service mediante una operación transaccional. No debe depender únicamente de Angular.

15. Migraciones y seed

1. Definir schema.prisma
2. Crear migración inicial
3. Generar Prisma Client
4. Crear seed de categorías/oficios
5. Crear datos de prueba
6. Probar restricciones
7. Versionar migraciones

El seed garantiza que todos los integrantes trabajen con el mismo catálogo.

16. Datos fuera de alcance

Elemento	Tabla	Razón
Pagos	No	Acuerdo externo.
Tarjetas	No	Sin procesamiento financiero.
Chat/mensajes	No	WhatsApp/teléfono.
Facturas	No	No solicitado.
Comisiones	No	Sin intermediación financiera.
Offline	No	Excluido.

17. Extensiones futuras

Extensión	Propósito
asignaciones	Registrar trabajador que toma una publicación.
favoritos	Guardar perfiles/publicaciones.
notificaciones	Avisos.
reportes	Moderación.
ubicaciones	Coordenadas y distancia.
media	Fotos y portafolio.
historial_estados	Auditoría.
verificaciones	Identidad/experiencia.

18. Casos de uso

18.1 Registro de trabajador

POST /auth/register → DTO → unicidad → Bcrypt → Usuario → PerfilTrabajador → tokens

18.2 Crear publicación

JWT → cliente → rol CLIENTE → DTO → oficio activo → Publicacion ABIERTA

18.3 Buscar trabajadores

GET /trabajadores → filtros oficio/zona/disponibilidad → joins → paginación → datos públicos

18.4 Crear reseña

JWT → cliente → validar publicación → verificar propietario → verificar elegibilidad → Resena 1..5

19. Checklist

- ☐ PostgreSQL + Prisma definidos.
- ☐ schema.prisma y migración inicial.
- ☐ UUID y enums.
- ☐ Seed de categorías/oficios.
- ☐ PrismaService.
- ☐ Bcrypt.
- ☐ JWT y Guards.
- ☐ Roles y propiedad.
- ☐ Perfil trabajador.
- ☐ trabajador_oficios.
- ☐ Máximo 3 principales.
- ☐ Publicaciones.
- ☐ Filtros y paginación.
- ☐ Estados.
- ☐ Reseñas 1..5.
- ☐ Promedio/cantidad.
- ☐ Índices.
- ☐ Swagger.
- ☐ ValidationPipe.
- ☐ No exponer password_hash.
- ☐ Errores consistentes.
- ☐ Datos de prueba.
- ☐ Pruebas de relaciones.
- ☐ DTOs y respuestas documentados.
- ☐ Medición de consultas.

20. Conclusiones y recomendaciones

El modelo cubre el alcance actual sin introducir entidades innecesarias. La separación entre cuentas y perfiles profesionales mantiene limpia la autenticación; categorías y oficios permiten filtros; trabajador_oficios resuelve la relación N:M; publicaciones representan solicitudes; y reseñas construyen reputación.

La principal decisión de alcance es que no existe todavía una contratación formal. Si el proyecto necesita registrar qué trabajador tomó un servicio, debe añadirse una entidad específica de asignación, no reutilizar campos ambiguos de publicaciones.

WhatsApp y las llamadas no requieren tablas adicionales: el Backend obtiene el teléfono del usuario y genera los enlaces de contacto.

Arquitectura final: Angular → HTTPS/JWT → NestJS → Controllers → Services → Prisma → PostgreSQL. schema.prisma, DTOs, enums y reglas de negocio deben ser el contrato común entre Backend y Frontend.

Resultado: una base normalizada, segura y preparada para búsquedas por oficio, categoría, ubicación, disponibilidad y estado, además de perfiles, publicaciones y reputación.